Lesson 4:
Error Analysis — Finding Failures

Shreya Shankar & Hamel Husain

Last time: synthetic data generation

- Generate dimensions of the problem, hypothetical but realistic values for each dimension, and then synthetic requests based on each tuple.
- Confusion around "expected outcome"
- Better phrase: extra metadata

Example tuple

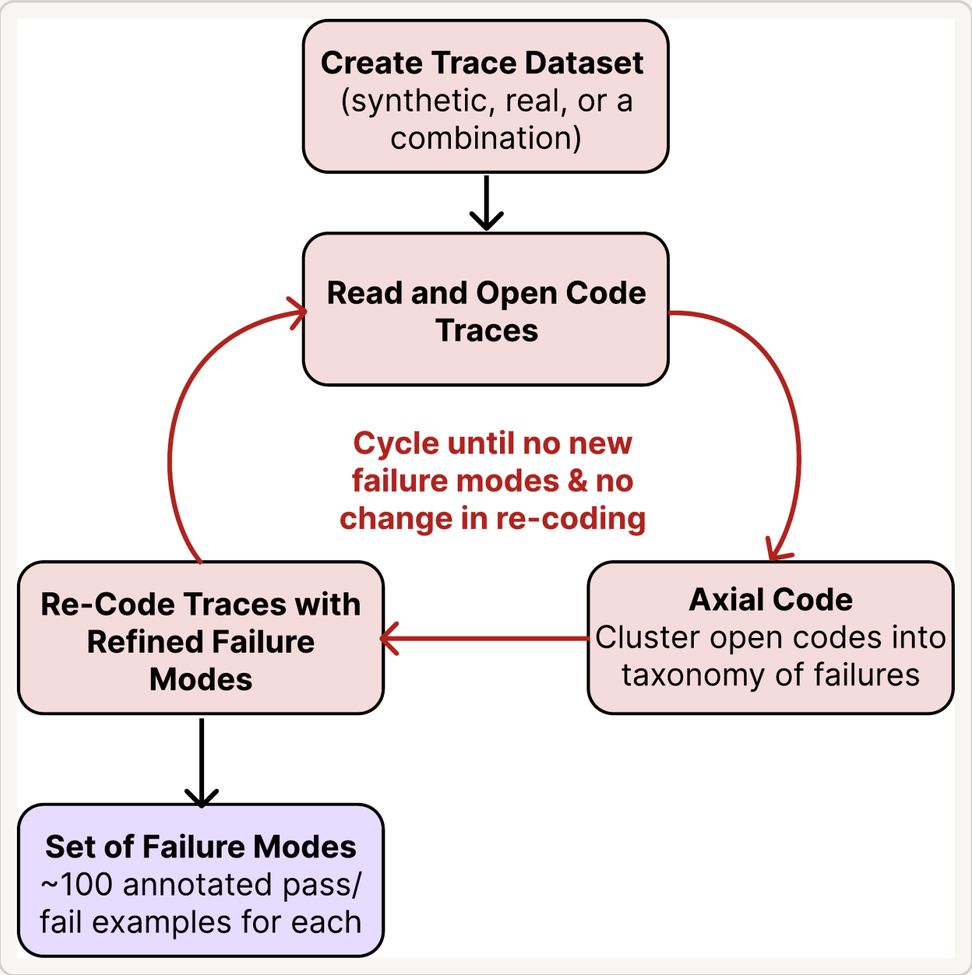
```
(role = shopper,
 intent = return,
 days_since_delivery = 45,
 return_window_days = 30,
 store_override = none)
```

Generated request

"Can I return the vase I got a while ago? I never used it."

45 days since delivery is absent from the request. Show the delivery date and the 30 day return policy in the error analysis UI to judge whether the agent correctly declined the return.

Today



- Open coding: read traces, write notes
- Axial coding: group notes into failure modes
- Output: 5-8 categories, with counts

Live: open coding

Build a review UI with the error-discovery skill, then read traces and write notes

Live: axial coding

Group notes into categories, merge and split until stable

Sampling traces from the firehose

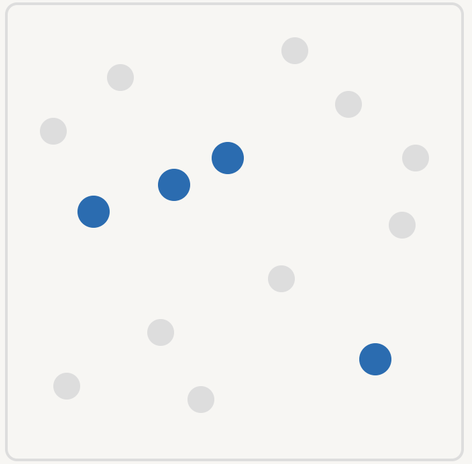
In production you may have thousands of traces. Which ones do you look at?

- Uniform random
- Cluster representatives
- Dimension targeted
- Depth search

Method 1: Uniform random

Pick traces at random. Every trace has an equal chance of being selected.

```
sample = random_sample(all_traces,
k=N)
for trace in sample:
    read(trace)
    annotate_first_failure(trace)
```



grey = all ● blue = sampled

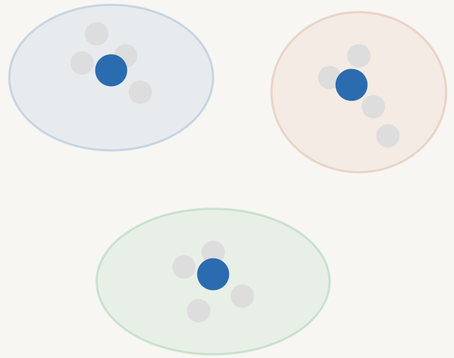
- Pro: simple, unbiased baseline
- Con: most traces pass, so you read many to find few failures

Method 2: Cluster representatives

- For each trace, compute: turn count, tool call count, distinct tools used, has retrieval, user role (one-hot), total tokens
- Cluster traces by these features (k-means), split your budget equally across clusters
- Pro: covers structural variety random misses
- Con: clusters reflect structure, not failure modes

```
clusters = kmeans(features, k=8)
per_cluster = budget // k

for cluster in clusters:
    pick per_cluster traces
    nearest the centroid
```



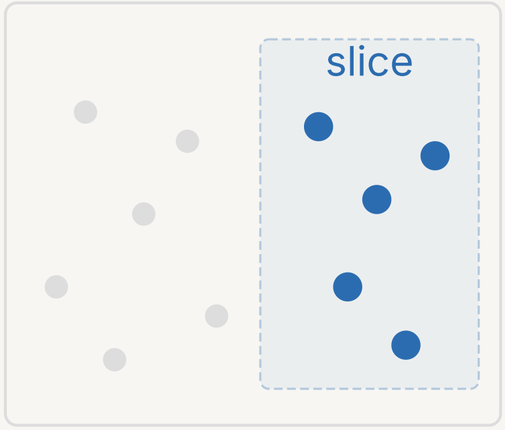
● picked from each cluster

Method 3: Dimension targeted

- Choose one dimension before looking at outcomes, e.g., all store override scenarios, or all merchant traces
- Review every trace in that slice
- Pro: finds failures concentrated in one part of the specification
- Con: only covers one dimension at a time

```
# all store-override scenarios
slice = filter(traces,
    policy == "store_override")

for trace in slice:
    read(trace)
    annotate_first_failure(trace)
```



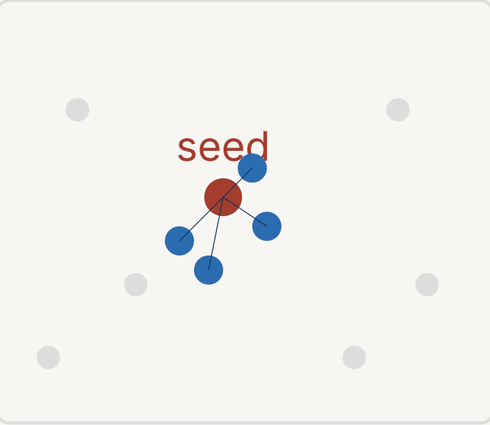
Method 4: Depth search

- Start from one confirmed failure
- Find traces with similar text (bag of words or embeddings)
- Review each candidate: same failure, or not?
- Pro: efficiently finds more instances of a known problem
- Con: only finds what you already know about

```
seed = confirmed_failure_trace

similar = find_nearest(
    seed, all_traces, k=25)

for trace in similar:
    read(trace)
    same failure? annotate it
```



Error analysis over time

Failure mode	v1	v2	v3
Invented return policy	18%	12%	~~3%~~ ✓
Refund without confirmation	12%	12%	11%
Pending reported as complete	8%	8%	~~2%~~ ✓
Wrong store's orders shown	—	5%	4%
Dropped multi-turn context	—	—	7%

- Sample new production traces, review them weekly
- Merge and rebalance within the existing taxonomy as modes get fixed
- Every few months, redo the taxonomy from scratch

Similar challenges described in Teresa Torres' Opportunity Solution Tree

Homework 4 and before Lesson 5

- **A** Build a review interface with a coding agent
- **B** Review at least 100 traces, open code each one
- **C** Use Raindrop Workshop to find modes you missed
- **D** Construct 5 to 8 failure modes with definitions, examples, boundaries
- **E** Apply every mode to every reviewed trace

Homework 4 instructions

When	What
Wed 9/16, 9:30 AM PT	Office hours
Sat 9/19, 11 AM PT	Lecture 5: Measuring with Evaluators

Please ask questions in Discord!